

バーPOS 技術仕様書

開発者・システム担当者向けの仕様書です。計算ロジック、データ構造、基盤、セキュリティ、既知の制約までを記載します。

1. システム概要

iPad のブラウザ上で動作するバー向け POS（注文・会計・売上管理）です。売上データは Firebase Firestore にリアルタイム保存され、オーナーは別端末（PC等）で同じ URL を開くだけで最新の売上を確認できます。

- 注文入力（席ごと、品目単位で担当キャストを設定。席と未会計注文は Firestore に保存され、リロードしても復元・端末間で共有）
- 会計（現金 / カード / QR、お釣り計算、決済手数料・実入金額の自動計算）
- 売上管理（期間集計、支払方法別、キャストバック集計、CSV出力）
- メニュー / キャスト / 手数料 / バック率 / 消費税をオーナーが画面から管理
- バックは2系統: 卓バック（合計×卓バック率→卓の担当キャスト・複数なら頭割り） + キャストドリンクバック（ドリンク料金×率→その担当へ上乗せ）
- 消費税は「税込で登録・加算しない」 / 「税抜で登録・会計時に加算」を切替 + 税率設定可
- スタッフ / オーナーの権限分離（ログイン）

2. 技術スタック・基盤

項目	内容
フロントエンド	React 18 + TypeScript + Vite 5
状態管理	Zustand 4
認証	Firebase Authentication（メール/パスワード）
データベース	Firebase Firestore
ホスティング	Firebase Hosting
対象端末	iPad Safari（PC ブラウザでも動作）
アイコン	Tabler Icons（CDN 読み込み）

- Firebase プロジェクト ID: `bar-pos-b1993`
- 公開 URL（デプロイ後）: `https://bar-pos-b1993.web.app`
- すべてクライアントサイド SPA。サーバーサイドのコードは無し（バックエンドは Firebase の各サービス）。

構成イメージ

```
[iPad Safari] ─┐
                │└─> Firebase Hosting (静的SPA配信)
[PC ブラウザ] ─┐
                │
                │└─> Firebase Authentication (ログイン/ロール判定)
                └─> Firestore (メニュー/キャスト/設定/席・注文/取引をリアルタイム同期)
```

3. ディレクトリ構成

```
src/
├── components/
│   ├── LoginScreen.tsx      # ログイン画面
│   ├── OrderScreen.tsx      # 注文入力 (席・メニュー・伝票)
│   ├── CheckoutScreen.tsx   # 会計
│   ├── SalesScreen.tsx      # 売上管理 (オーナー専用)
│   ├── MenuManageScreen.tsx # メニュー管理 (オーナー専用)
│   └── CastManageScreen.tsx # キャスト管理 (オーナー専用)
├── hooks/
│   └── useSalesSummary.ts    # 売上・バック集計ロジック
├── lib/
│   ├── firebase.ts          # Firebase 初期化 (Firestore + Auth) ・コレクション定数
│   ├── authConfig.ts        # ログインID⇄メール変換・ロール判定
│   ├── tax.ts               # 税・手数料の計算ユーティリティ
│   ├── csv.ts               # CSV 生成・ダウンロード
│   └── defaultMenus.ts      # デフォルトメニュー / フリー入力プリセット
├── store/
│   └── posStore.ts           # Zustand グローバル状態・全アクション
├── types/index.ts            # 型定義
├── styles/global.css         # スタイル
├── App.tsx                   # ルート (認証ゲート・画面切替・購読)
├── main.tsx                  # エントリーポイント
└── firestore.rules            # Firestore セキュリティルール
```

4. データモデル (Firestore)

コレクション名は `src/lib/firebase.ts` の `COLLECTIONS` で定義。

`transactions` (会計確定済みの取引)

会計確定時に1件追加 (追記のみ。更新・削除はアプリからは行わない)。

フィールド	型	説明
seatName	string	席名（未入力時は「席 A」等）
solo	boolean	一人客フラグ
items	OrderItem[]	注文品目の配列（各品目に担当キャスト名を保持）
subtotal	number	税抜合計
tax	number	消費税額
total	number	税込合計
payMethod	'cash' 'card' 'qr'	支払方法
feeRate	number	決済手数料率（%）
feeAmount	number	決済手数料額
netAmount	number	実入金額（total – feeAmount）
primaryCast	string	主担当キャスト（売上最大の担当。CSV表示用）
tableCasts	string[]	卓バックの受取キャスト（卓の担当。複数なら頭割り）
completedAt	number	会計時刻（Unix ミリ秒）
_createdAt	Timestamp	サーバータイムスタンプ（書き込み時刻）

OrderItem（items 配列の要素）：

フィールド	型	説明
id	string	一意ID
name	string	品名
priceExTax	number	単価（税込モード時はその額が最終価格）
qty	number	数量
cast	string	担当キャスト名（キャストドリンクのドリンクバック用。空文字=未設定）
category	string	カテゴリ（キャストドリンク判定等。フリー入力は 'フリー入力' ）
isToday	boolean	本日限定メニュー
isFree	boolean	フリー入力

menus（メニュー）

{ name, priceExTax, category, isToday, sortOrder }。sortOrder 昇順で表示。本日限定は isToday: true ・ category: '本日限定'。カテゴリは MENU_CATEGORIES（src/lib/defaultMenus.ts）：セット / ショット / シャンパン / キャストドリンク / ウイスキー / カクテル / ビール / フード。

casts（キャスト）

{ name, sortOrder }。sortOrder 昇順で表示。注文画面の担当選択肢になる。

tables (席・未会計の注文。ドキュメントID=席ID)

{ name, solo, tableCasts: string[], items: OrderItem[], createdAt, updatedAt }。tableCasts は卓の担当キャスト (複数可)。

- 注文操作のたびに該当席のドキュメントを保存 (ローカル楽観更新+Firestore書き込み)。
- ログイン後に購読してハイドレートするため、リロードしても復元・端末間で共有。
- 会計確定で items を空にする (席は残して再利用)。初回起動時は席 A/B/C を自動作成 (固定IDなので重複しない)。

settings (設定。ドキュメントIDで区別)

- settings/fees ... { card: number, qr: number } (各%)
- settings/back ... { rate: number, drinkRate: number } (卓バック率・キャストドリンクバック率。0.10 = 10%)
- settings/tax ... { rate: number, mode: 'exclusive' | 'inclusive' } (消費税率と税の扱い)

5. 認証・権限

ログイン方式

Firebase Authentication の「メール/パスワード」を使用。現場で打ちやすいよう、ログイン画面では **ユーザーID** だけを入力し、内部でメールアドレスへ変換する (src/lib/authConfig.ts)。

- 変換: ID → ID@<VITE_AUTH_ID_DOMAIN> (既定ドメイン bar-pos.local)
- 例: ログインID owner → owner@bar-pos.local

ロール判定

ログイン中のメールアドレスで自動判定 (DB にロールを持たせない)。

```
role = (email === `${VITE_OWNER_ID}@${VITE_AUTH_ID_DOMAIN}`) ? 'owner' : 'staff'
```

- owner@bar-pos.local → **オーナー** (全画面)
- それ以外 → **スタッフ** (注文入力・会計のみ)

アプリ側 (画面の出し分け) と Firestore ルール (DB アクセス制御) の **両方** が同じメールで判定するため、UI を隠すだけでなく DB レベルでも保護される。

Firestore セキュリティルール (firestore.rules)

コレクション	read	write
menus	ログイン済み	オーナーのみ
casts	ログイン済み	オーナーのみ
settings	ログイン済み	オーナーのみ
tables	ログイン済み	ログイン済み (スタッフも注文を取るため読み書き可)
transactions	オーナーのみ	create はログイン済み (=スタッフも会計確定可)、update/delete はオーナーのみ

→ スタッフのアカウントでは (UI だけでなく) DB から直接でも過去売上を読めない。

ルール内のオーナー判定は `owner@bar-pos.local` をハードコードしている。`VITE_OWNER_ID` / `VITE_AUTH_ID_DOMAIN` を変えたら、ルールの該当箇所も `<VITE_OWNER_ID>@<VITE_AUTH_ID_DOMAIN>` に合わせることを。

6. 計算ロジック

実装は `src/lib/tax.ts` (税・手数料)、`src/store/posStore.ts` (会計確定)、`src/hooks/useSalesSummary.ts` (集計)。

6.1 消費税 (税率・税の扱いは設定可能)

消費税率 (`settings.tax.rate`、既定0.10) と「税の扱い」(`settings.tax.mode`) をオーナーが画面から設定する。実装は `calcBill(base, rate, mode)` (`src/lib/tax.ts`)。 `base = \sum(priceExTax \times qty)`。

exclusive (税抜で登録・会計時に加算) 明細は税抜で表示し、消費税は小計に対して1回だけ計算 (端数切り捨て)。「明細の合計 = 小計 = 合計欄」が一致する。

```
subtotal = base
tax       = floor(base \times rate)
total    = subtotal + tax
```

メニューボタンの単価表示は税込 (`displayUnit = floor(price \times (1+rate))`) = お客様向け総額表示。

inclusive (税込で登録・加算しない) 登録額がそのまま最終価格。消費税は加算も表示もしない。

```
subtotal = base
tax       = 0
total    = base
```

この場合、メニュー単価表示は登録額そのまま (`displayUnit = price`)。注文/会計画面の小計・消費税行は非表示。

例 (exclusive・税率10%) : 税抜 545 円 $\times$ 3 $\rightarrow$ subtotal 1,635 / tax 163 / total 1,798 例 (inclusive) : 税込 599 円 $\times$ 3 $\rightarrow$ total 1,797 (消費税の行は出さない)

6.2 決済手数料・実入金額

```
feeRate = (cash: 0, card: settings.fees.card, qr: settings.fees.qr) // %
feeAmount = floor(total \times feeRate / 100)
netAmount = total - feeAmount
```

既定値: カード 3.25%、QR 1.98% (オーナーが画面から変更可)。現金は手数料 0。

例: total 10,000 $\cdot$ カード 3.25%

```
feeAmount = floor(10,000 \times 0.0325) = 325
netAmount = 9,675
```

6.3 お釣り（現金）

```
change = cashReceived - total    // cashReceived ≥ total のときのみ確定可能
```

預かり金額は `total` 以上のプリセット（1,000/2,000/5,000/10,000+ぴったり額）から選択。

6.4 キャストバック（2系統：卓バック+キャストドリンクバック）

実装は `useSalesSummary`。設定は `settings/back` の `rate`（卓バック率）と `drinkRate`（ドリンクバック率）。

① **卓バック**：会計の合計に卓バック率をかけ、卓の担当キャスト（`tableCasts`）で**頭割り**。

```
n = tableCasts.length
各 tableCast へ: backRaw += (total × rate) / n
(担当が0人の卓はバック対象外＝未設定)
```

② **キャストドリンクバック**：`category === 'キャストドリンク'` の品目について、料金 × ドリンク率を**その品目の担当**（`item.cast`）へ**上乘せ**。

```
そのドリンクの担当へ: backRaw += (priceExTax × qty) × drinkRate
```

- キャストドリンクの料金は**卓バックの合計にも含まれる**ため、卓の担当とドリンクの担当の**両方**に効く（二重計上＝仕様）。
- 最後に各キャストの `backRaw` を四捨五入して `backAmount`。
- 担当未設定はバック対象外（0円）。集計には可視化。

例（卓バック10% / ドリンクバック率はお店の設定。卓の担当=たま）：

```
セット 2,000 + キャストドリンク（りな）1,200 = 合計3,200
→ たま（卓バック）：3,200 × 10% = 320
→ りな（ドリンクバック）：1,200 × ドリンク率
卓の担当が2名なら卓バックは頭割り（320 → 160ずつ）
```

6.5 売上サマリー（`useSalesSummary`）

期間内の `transactions` から集計：

```
totalSales    = Σ total          // 税込
totalFee      = Σ feeAmount
totalNet      = totalSales - totalFee
txCount       = 取引件数
avgPerCustomer = round(totalSales / txCount)    // 1取引あたり（人数ベースではない）
soloCount     = solo の件数
byMethod[m]   = { count, sales(=Σtotal), fee, net } を支払方法別に集計
castSummaries = 6.4 の結果（卓数 / 卓売上+ドリンク売上 / 卓バック+ドリンクバック）
```

注: キャスト別「売上」は卓担当の卓売上+担当ドリンク料金で、キャストドリンクは卓担当・ドリンク担当に二重計上され得るため、上部「売上合計」とは一致しない（仕様）。

6.6 期間の範囲 (`SalesScreen.periodRange`)

期間	from	to
今日	当日 00:00:00	当日 23:59:59.999
今週	6日前の 00:00:00 (直近7日のローリング)	当日 23:59:59.999
今月	当月1日 00:00:00	当日 23:59:59.999

「今週」はカレンダー週ではなく直近7日である点に注意。

7. リアルタイム同期

`onSnapshot` で購読:

- `menus` ・ `casts` ・ `tables` : ログイン後に全ユーザーが購読。 `tables` はハイドレートして席・注文を復元 (リロード耐性・端末間共有)。
- `transactions` : 売上画面 (オーナー) で、選択期間を `where(completedAt >= from && <= to)` で絞り込み購読 (全件取得はしない)。

注文操作はローカル楽観更新+該当 `tables` ドキュメントへ書き込み (fire-and-forget)。会計確定は `transactions` へ `addDoc` (追記) し、その席の `items` を空にする。表示中の画面と選択中の席は `localStorage` に保存し、リロード後に同じ画面・同じ席へ復元する (端末ローカルのUI状態)。

8. CSV 出力 (`src/lib/csv.ts`)

- BOM 付き UTF-8 (Excel で文字化けせず開ける)。
- 売上CSV: 日時/席名/一人客/支払方法/税抜売上/消費税/税込売上/手数料率/手数料額/実入金額/主担当キャスト。
- バックCSV: キャスト/担当件数/売上合計(税抜)/バック額。

9. 環境変数 (`.env`)

```
VITE_FIREBASE_API_KEY=...
VITE_FIREBASE_AUTH_DOMAIN=...
VITE_FIREBASE_PROJECT_ID=...
VITE_FIREBASE_STORAGE_BUCKET=...
VITE_FIREBASE_MESSAGING_SENDER_ID=...
VITE_FIREBASE_APP_ID=...
VITE_AUTH_ID_DOMAIN=bar-pos.local # ログインID→メール変換ドメイン
VITE_OWNER_ID=owner # オーナー扱いにするログインID
VITE_BACK_RATE=0.30 # バック率の初期値 (画面保存値が優先)
```

`.env` は `.gitignore` 済み (Firebase キーを含むためリポジトリに含めない)。

10. 既知の制約・注意点

実運用前に把握しておくべき項目（現時点で未対応）。

1. **取引の取消・返金機能が無い**。打ち間違いはオーナーが Firestore を直接修正する。
2. **オフライン耐性が限定的**。会計確定時にネットワーク失敗してもアプリ上に明確なエラー表示が出ない（Firestore SDK のキュー任せ）。Firestore のオフライン永続化（`persistentLocalCache`）も未設定のため、完全オフラインでのリロードは未保証。
3. **CSV インジェクション対策なし**。席名等に `=`, `+`, `-`, `@` で始まる文字列を入れると、Excel で数式として解釈され得る。
4. **「今週」は直近7日**（カレンダー週ではない）。
5. **客単価は1取引あたり**（来店人数ベースではない）。
6. **キャストのリネーム/削除は過去データに遡及しない**。取引内には会計時点のキャスト名（文字列）が保存されるため、改名後も過去記録は旧名のまま。
7. **席・注文の競合**: 同じ卓を2端末で同時編集した場合は最後の書き込みが優先（小規模運用を想定）。

補足: 「席・未会計の注文がリロードで消える」問題は `tables` コレクションへの永続化で解消済み（オンライン時）。

11. セットアップ・デプロイ

詳細は [README.md](#) を参照。要点のみ:

```
npm install
cp .env.example .env # Firebase 設定値とログイン設定を記入
npm run dev          # http://localhost:5173

# デプロイ
npm run build
firebase deploy      # Hosting (firebase.json 設定済み)
```

- Firestore ルールは `firestore.rules` の内容を Firebase Console → Firestore → ルール に貼って「公開」（または CLI で反映する場合は `firebase.json` に firestore 設定を追加）。
- Authentication で「メール/パスワード」を有効化し、`owner@bar-pos.local` / `staff@bar-pos.local` を作成。
- メニュー・キャストの初期データは、オーナーでログインして各管理画面の「投入」ボタンから登録。

12. 今後の改善候補

- 取引の取消・返金フロー
- オフライン永続化（`persistentLocalCache`）と会計失敗時のエラー表示
- CSV インジェクション対策（先頭エスケープ）
- 真の PWA 化（manifest / Service Worker）